

Project Experience: Mission Control – Multi-Soldier Task Orchestrator

What I Learned

This project helped me understand the complete lifecycle of building distributed systems.

I learned to design and deploy a multi-service architecture using FastAPI, RabbitMQ, Redis, and Docker — ensuring smooth communication between components through queues and tokens.

It also gave me hands-on exposure to:

- JWT authentication and token rotation for secure communication.
- Redis as a mission data store and per-soldier queue manager.
- RabbitMQ for asynchronous message exchange between Commander and Soldiers.
- Docker Compose for running a fully containerized environment.

Overall, I developed a deeper understanding of system orchestration, microservice interaction, and debugging distributed workflows.

How I Approached the Problem

I began by visualizing the problem as a mission management system:

- A Commander service assigns tasks and manages mission states.
- Multiple Soldier workers receive and execute those missions asynchronously.
- A message queue (RabbitMQ) ensures decoupled communication between them.
- Redis stores mission progress and queue information for each soldier.

I focused on building the project incrementally — starting with simple APIs, then introducing JWT-based security, and finally containerizing everything.

Automated scripts (.ps1 and .sh) were added later to simulate multiple mission assignments and monitor real-time progress.

3 Challenges & How I Solved Them

This project came with multiple challenges that improved my debugging and architectural thinking:

- **RabbitMQ Connection Failures:**
Solved by using Docker service hostnames (e.g., rabbitmq, commander) instead of localhost.
- **Redis WRONGTYPE Errors:**
Occurred when different data types shared the same key. Fixed by defining clear key patterns like mission:{id} and soldier_queue:{id}.
- **Queued Tasks Not Executing:**
Fixed by implementing a per-soldier queue mechanism in Redis and enhancing the worker logic to pick the next mission automatically after completing the previous one.
- **Token Refresh Overwrites:**
Adjusted token caching so that tokens only refresh after expiry instead of generating new ones on every request.

These issues taught me how to trace and fix problems across multiple containers effectively.

4 Key Takeaways

This project strengthened my ability to design, debug, and optimize distributed systems. I learned how crucial structured logging, retry mechanisms, and proper container networking are for stable microservice communication.

More importantly, it taught me how to think like a systems engineer — not just writing code, but making components collaborate reliably under real-world conditions.